

LESSON 7

Managing a small datasets in Python

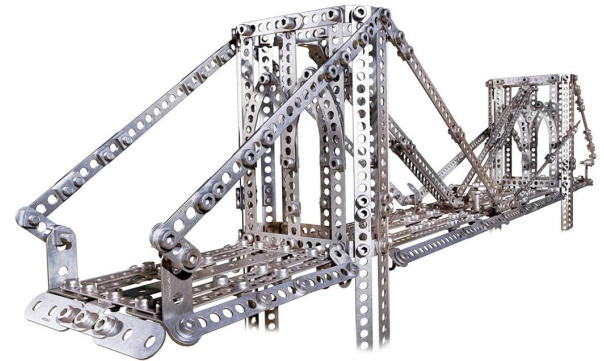
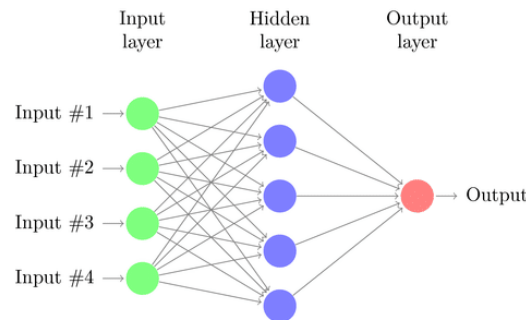
Degrees of freedom/parameters

Data Leakage



Outline

- Laboratory in **Python/Colab** on training data management
- Number of degrees of freedom/parameters
- Data Leakage
- Main points

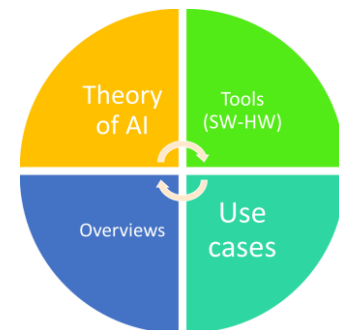




Toolboxes

Laboratory

Python/Colab



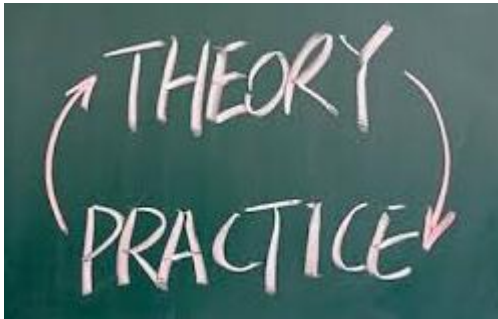
Laboratory outline

- **LABORATORY (in a separate file):**
Loading and basic dataset management
 - Colab example
 - Matlab example
- The laboratory is based on the IRIS dataset
- **Please refer to the next lessons about the description of the IRIS dataset**
 - Classify iris flowers into 3 classes:
 1. Setosa
 2. Versicolor
 3. Virginica
 - 4 features:
 1. Sepal length in cm
 2. Sepal width in cm
 3. Petal length in cm
 4. Petal width in cm
 - 150 samples



What in the exam?

- In the exam there will be **no (exact) coding**
- Nevertheless, is it important to study the application of the concept of the theory in practical cases and examples in order to better understand the notions



Knowledge

- You can be asked **to comment/answer about some example** of code

LAB CLASS

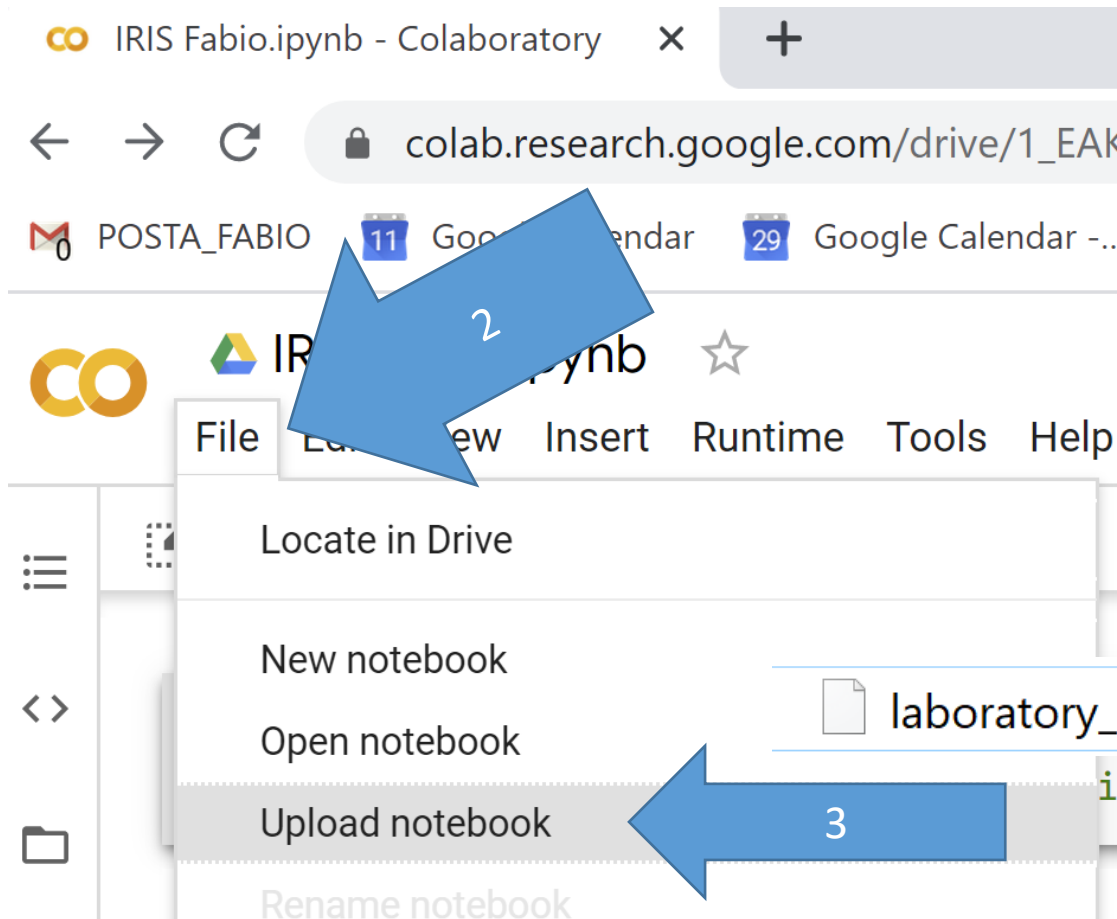
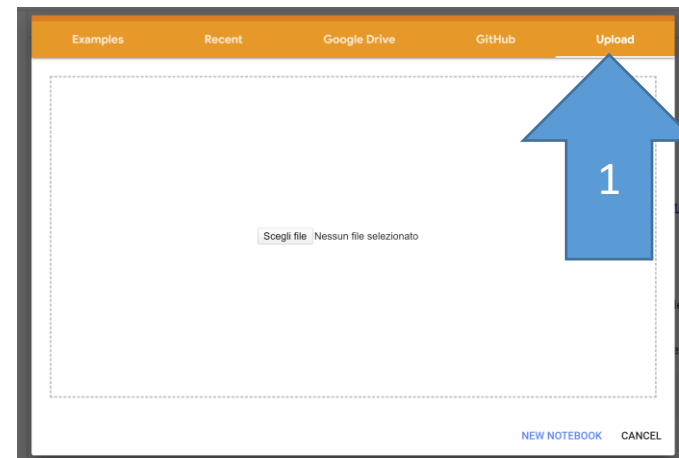
COLAB

- Environment preparation
- Data loading
- Preliminary data analysis
- Next lessons:
plot the data, train models,
compare models, etc.



Upload your *.ipynb

<https://colab.research.google.com/>



An **IPYNB** file is a notebook document used by Jupyter Notebook, an interactive computational environment designed to help designers to work with the Python language and their data.

Let's run it!

laboratory_COLAB_IRIS_Dataset.ipynb ☆

File Edit View Insert Runtime Tools Help Last saved at 10:39 PM

+ Code + Text

IRIS data loading, preparation, classification

Fabio Scotti fabio.scotti@unimi.it

Course: Intelligent systems for industry, supply chain and environment

A Matlab demo for

- loading a the IRIS dataset
- preparing the variables

[]

Import libraries

```
[ ] import pandas as pd #Python Data Analysis Library
import numpy as np #Python Scientific Library
from matplotlib import pyplot as plt
```

Load data

```
[ ] url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
new_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'iris_class']
dataset = pd.read_csv(url, names=new_names, skiprows=0, delimiter=',')
dataset.info()
```

1) Read the comments

1) Read the comments

Import libraries

```
[ ] import pandas
import numpy as np
from matplotlib import pyplot as plt
```

2



Import libraries

```
[ ] import pandas as pd #Python Data Analysis Library
import numpy as np #Python Scientific Library
from matplotlib import pyplot as plt
```

Click on the []
to run the cell

COLAB_IRIS_Dataset.ipynb ☆

Insert Runtime Tools Help

Run all Ctrl+F9

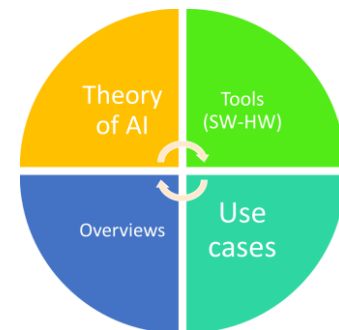
Run before Ctrl+F8

Or «Run all»



Toolboxes

Loading,
data wrangling/EDA
in **Python/Colab**



Environment preparation

Pandas: library for dataset preprocessing

Numpy: library for array manipulation

Import libraries

```
[74] import pandas as pd #Python Data Analysis Library  
import numpy as np #Python Scientific Library
```

NumPy (pronounced /'nʌmpaɪ/ (NUM-py) or sometimes /'nʌmpi/ (NUM-pee)) is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays.

Data loading

Iris dataset

- Use the panda library to read the CSV

Load data

```
[75] url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"  
     new_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'iris_class']  
     dataset = pd.read_csv(url, names=new_names, skiprows=0, delimiter=',')  
     dataset.info()
```

- Classify iris flowers into 3 classes:
 1. Setosa
 2. Versicolor
 3. Virginica
- 4 features:
 1. Sepal length in cm
 2. Sepal width in cm
 3. Petal length in cm
 4. Petal width in cm
- 150 samples



Data loading

`dataset.info()` gives us organized information

```
↳ <class 'pandas.core.frame.DataFrame'>  
RangeIndex: 150 entries, 0 to 149  
Data columns (total 5 columns):  
#   Column          Non-Null Count  Dtype  
---  -  
0   sepal_length    150 non-null    float64  
1   sepal_width     150 non-null    float64  
2   petal_length    150 non-null    float64  
3   petal_width     150 non-null    float64  
4   iris_class      150 non-null    object  
dtypes: float64(4), object(1)  
memory usage: 6.0+ KB
```

Data loading

`dataset.head(n)` to view the first n samples



	sepal_length	sepal_width	petal_length	petal_width	iris_class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa
8	4.4	2.9	1.4	0.2	Iris-setosa
9	4.9	3.1	1.5	0.1	Iris-setosa

Data preprocessing

First, we need to separate the features from the classes (target labels)

- Different preprocessing for features and classes (Necessary? YES! To avoid Data Leakage!!!)

```
[77] y = dataset['iris_class']  
     x = dataset.drop(['iris_class'], axis=1)  
  
print ("dataset : ",dataset.shape)  
print ("x : ",x.shape)  
print ("y : ",y.shape)
```



As size() in Matlab

Data preprocessing

First, we need to separate the features from the classes

- Feature matrix has 4 columns (the features)
- Target vector has 1 column (the class)

```
↳ dataset : (150, 5)  
   x : (150, 4)  
   y : (150, )
```

Data preprocessing

Use the numpy library to convert data into matrix format

- Ease of access to portions of the matrices

```
[78] xNumpy = x.to_numpy()  
     yNumpy = y.to_numpy()
```

Data preprocessing

We can access a single feature or a single samples

- Read a specific feature: matrix slicing by column

```
[79] feature_to_extract = 1  
     feature = xNumpy[:,feature_to_extract]  
     print(feature)
```

Data preprocessing

We can access a single feature or a single samples

- Read a specific feature: matrix slicing by **column**

```
[79] feature_to_extract = 1  
     feature = xNumpy[:,feature_to_extract]  
     print(feature)
```

```
☞ [3.5 3.  3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 3.7 3.4 3.  3.  4.  4.4 3.9 3.5  
   3.8 3.8 3.4 3.7 3.6 3.3 3.4 3.  3.4 3.5 3.4 3.2 3.1 3.4 4.1 4.2 3.1 3.2  
   3.5 3.1 3.  3.4 3.5 2.3 3.2 3.5 3.8 3.  3.8 3.2 3.7 3.3 3.2 3.2 3.1 2.3  
   2.8 2.8 3.3 2.4 2.9 2.7 2.  3.  2.2 2.9 2.9 3.1 3.  2.7 2.2 2.5 3.2 2.8  
   2.5 2.8 2.9 3.  2.8 3.  2.9 2.6 2.4 2.4 2.7 2.7 3.  3.4 3.1 2.3 3.  2.5  
   2.6 3.  2.6 2.3 2.7 3.  2.9 2.9 2.5 2.8 3.3 2.7 3.  2.9 3.  3.  2.5 2.9  
   2.5 3.6 3.2 2.7 3.  2.5 2.8 3.2 3.  3.8 2.6 2.2 3.2 2.8 2.8 2.7 3.3 3.2  
   2.8 3.  2.8 3.  2.8 3.8 2.8 2.8 2.6 3.  3.4 3.1 3.  3.1 3.1 3.1 2.7 3.2  
   3.3 3.  2.5 3.  3.4 3. ]
```

Data preprocessing

We can access
a single feature
or a single samples

- Read a specific feature: matrix slicing by column

```
[79] feature_to_extract = 1
      feature = xNumpy[:,feature_to_extract]
      print(feature)
```

Debugging!!

→ [3.5 3. 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 3.7 3.4 3. 3. 4. 4.4 3.9 3.5
2.8 2.8 3.4 2.7 2.6 2.2 2.4 2. 2.4 2.5 2.4 2.2 2.1 2.4 4.1 4.2 3.1 3.2
3.5 3.1 3. 3.4 3.5 2.3 3.2 3.5 3.8 3. 3.8 3.2 3.7 3.3 3.2 3.2 3.1 2.3
2.8 2.8 3.3 2.4 2.9 2.7 2. 3. 2.2 2.9 2.9 3.1 3. 2.7 2.2 2.5 3.2 2.8
2.5 2.8 2.9 3. 2.8 3. 2.9 2.6 2.4 2.4 2.7 2.7 3. 3.4 3.1 2.3 3. 2.5
2.6 3. 2.6 2.3 2.7 3. 2.9 2.9 2.5 2.8 3.3 2.7 3. 2.9 3. 3. 2.5 2.9
2.5 3.6 3.2 2.7 3. 2.5 2.8 3.2 3. 3.8 2.6 2.2 3.2 2.8 2.8 2.7 3.3 3.2
2.8 3. 2.8 3. 2.8 3.8 2.8 2.8 2.6 3. 3.4 3.1 3. 3.1 3.1 3.1 2.7 3.2
3.3 3. 2.5 3. 3.4 3.]

	sepal_length	sepal_width	petal_length	petal_width	iris_class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa
8	4.4	2.9	1.4	0.2	Iris-setosa
9	4.9	3.1	1.5	0.1	Iris-setosa

	sepal_length	sepal_width	petal_length	petal_width	iris_class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa
8	4.4	2.9	1.4	0.2	Iris-setosa
9	4.9	3.1	1.5	0.1	Iris-setosa

Data preprocessing

We can access a single feature or a single samples

- Read a specific sample: matrix slicing by **row**

```
[80] sample_to_extract = 1
      sample = xNumpy[sample_to_extract,:]
      print(sample)
```

Data preprocessing

We can access a single feature or a single samples

- Read a specific sample: matrix slicing by row

```
[80] sample_to_extract = 1  
     sample = xNumpy[sample_to_extract,:]   
     print(sample)
```

Debugging!!

➞ [4.9 3. 1.4 0.2]

	sepal_length	sepal_width	petal_length	petal_width	iris_class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa
8	4.4	2.9	1.4	0.2	Iris-setosa
9	4.9	3.1	1.5	0.1	Iris-setosa

Data preprocessing

We can access a
single feature
or a single samples

- Read a specific sample: matrix slicing by row

	sepal_length	sepal_width	petal_length	petal_width	iris_class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	5.0	3.2	1.5	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa
8	4.4	2.9	1.4	0.2	Iris-setosa
9	4.9	3.1	1.5	0.1	Iris-setosa

```
[80] sample_to_extract = 1
      sample = xNumpy[sample_to_extract,:]
      print(sample)
```

```
☞ [4.9 3.  1.4 0.2]
```

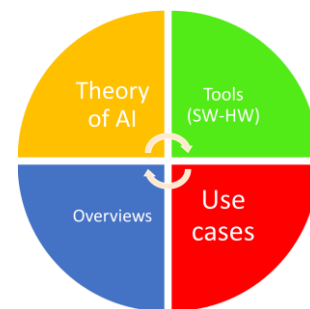


THEORY

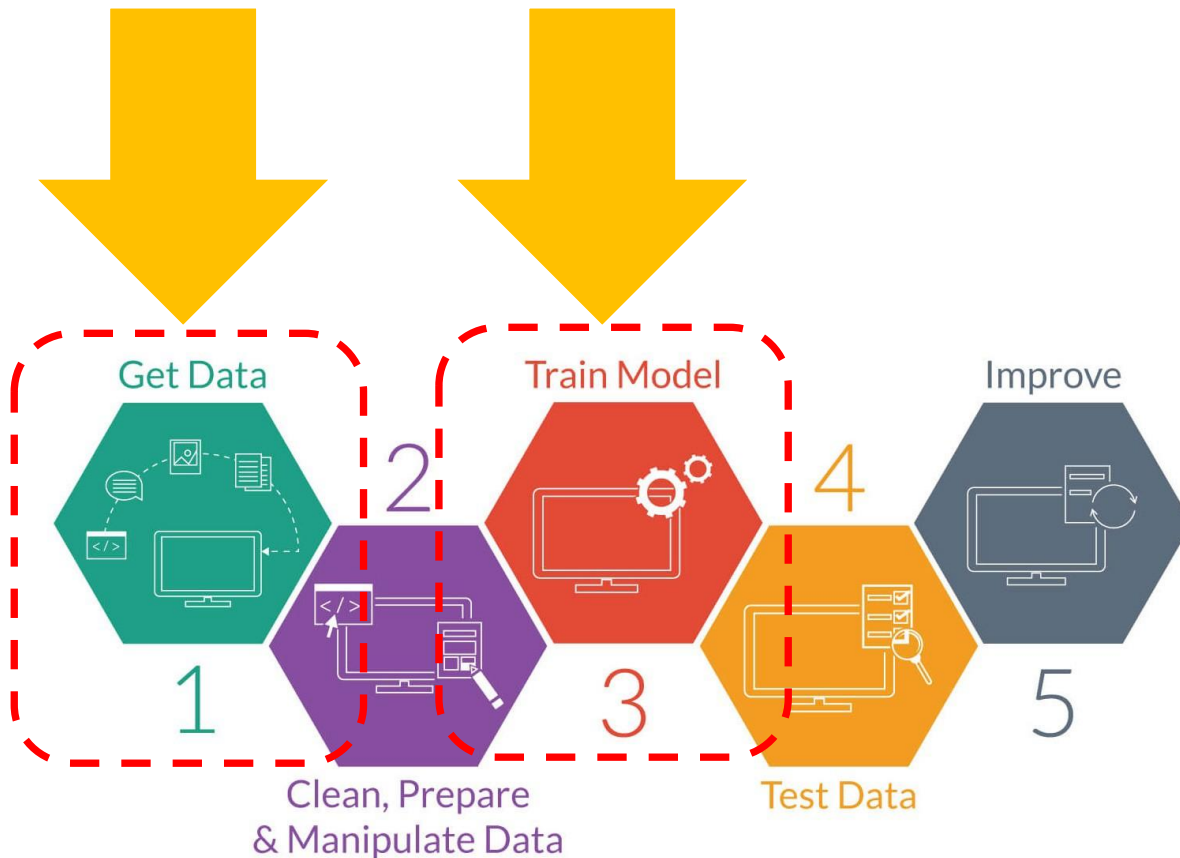
Degrees of freedom /parameters of the models

How much is complex your model?

How much data do you need to configure it?



Step 1 and 3 of the ML workflow

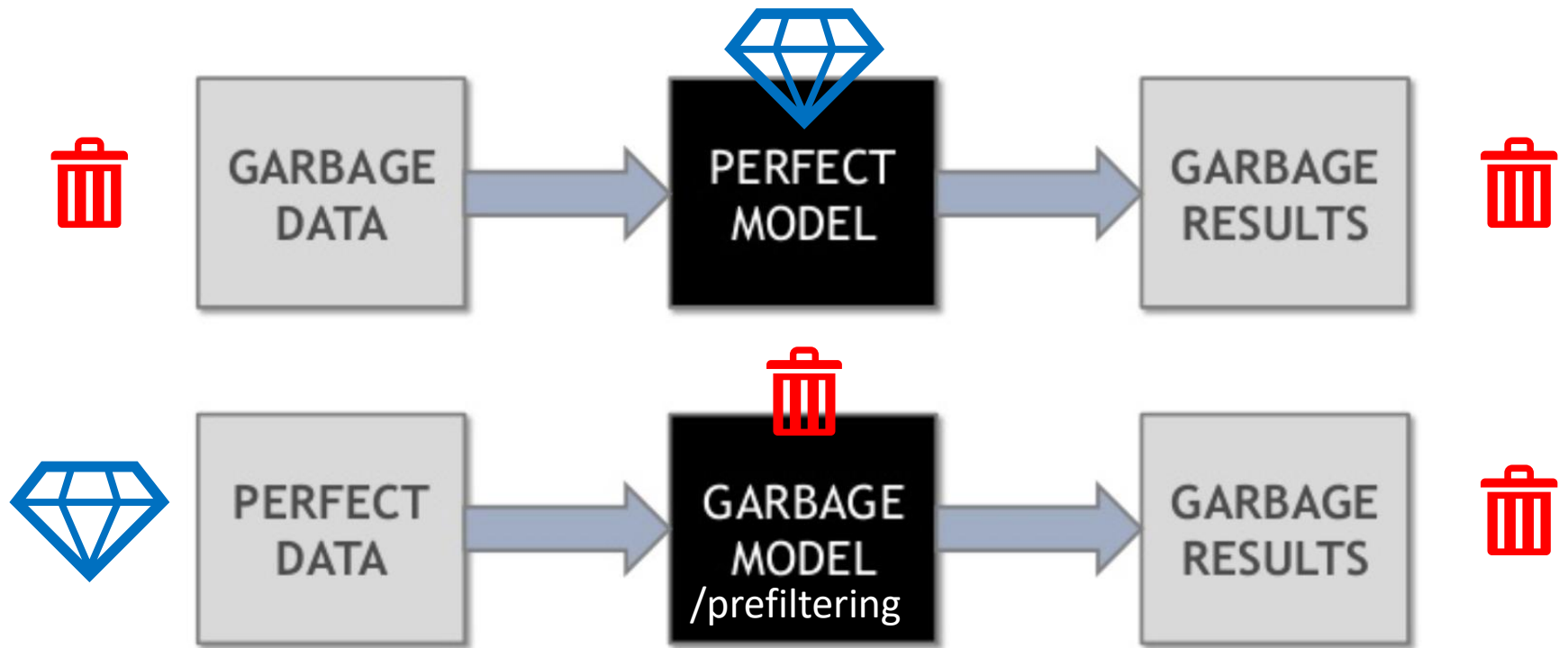


GIGO

NO «MAGICAL» NEURAL NETWORK
WILL SAVE YOU FROM THIS!!!



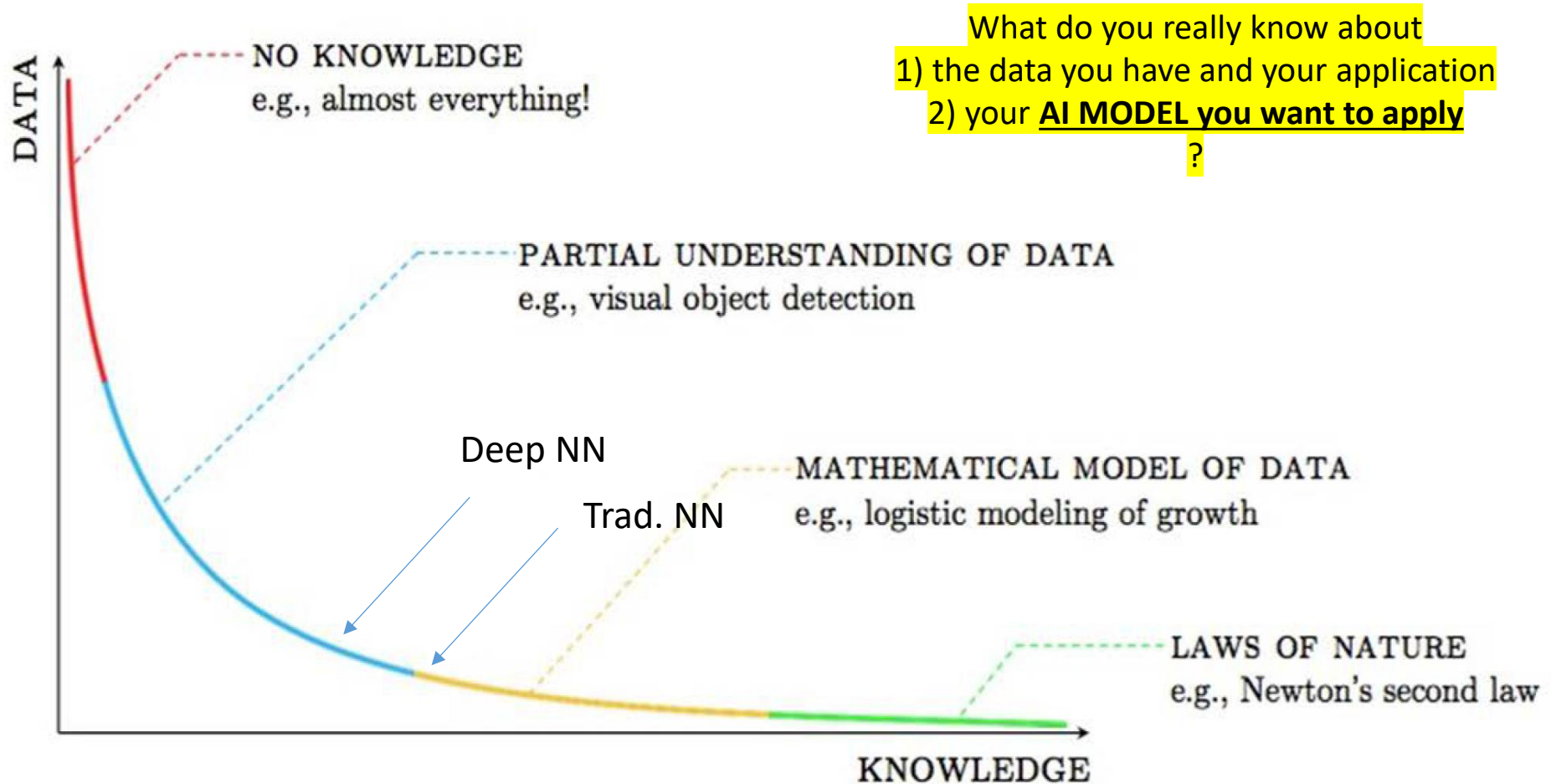
Even worse...



e.g., wrong prefiltering/feature engineering
wrong model choice,
badly trained NN,
overfitted NNs, etc.

Now is important to remember....

Data knowledge spectrum



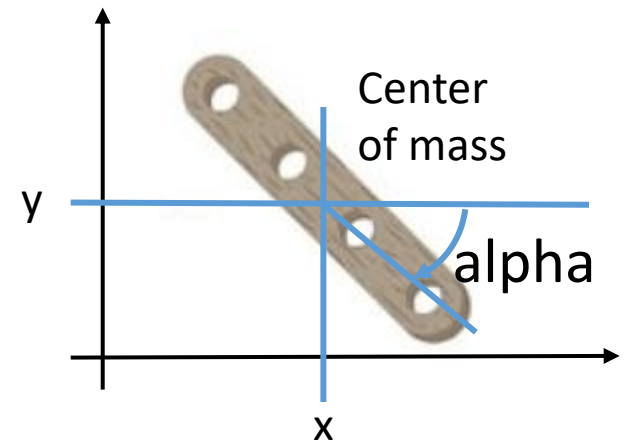
How much data?

Degree of freedom/parameters

- Imagine a small mechanical component with its **Degrees of Freedom (DoF)** or Number of Parameters (**#Par**) in a 2D space

- x ,
- y ,
- α

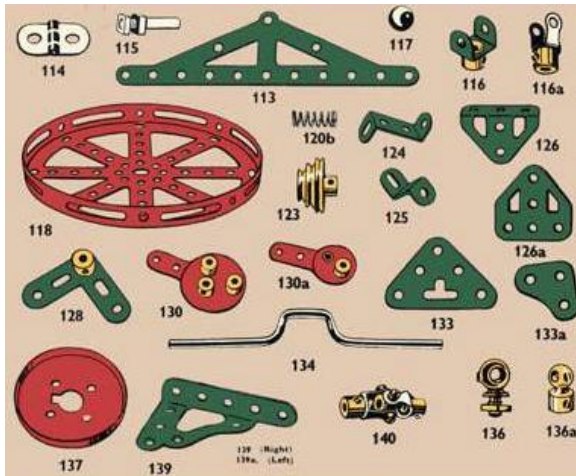
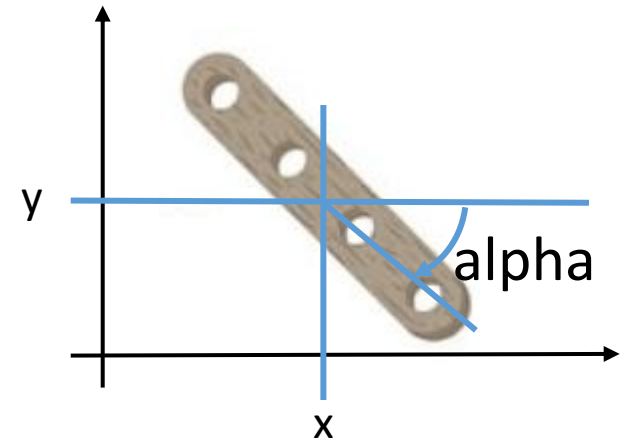
- In **physics**, the degree of freedom of a mechanical system is the number of independent parameters that define its configuration.
- Note: **DoF** is not exactly equivalent **#Par** for complex systems, but they are strongly related.



How much data?

DoF/#Par

- Image a small set of components, each of them with its parameters



DoF/parameters (2)

- A mechanical model (like a neural network) is a set of interconnected elements.

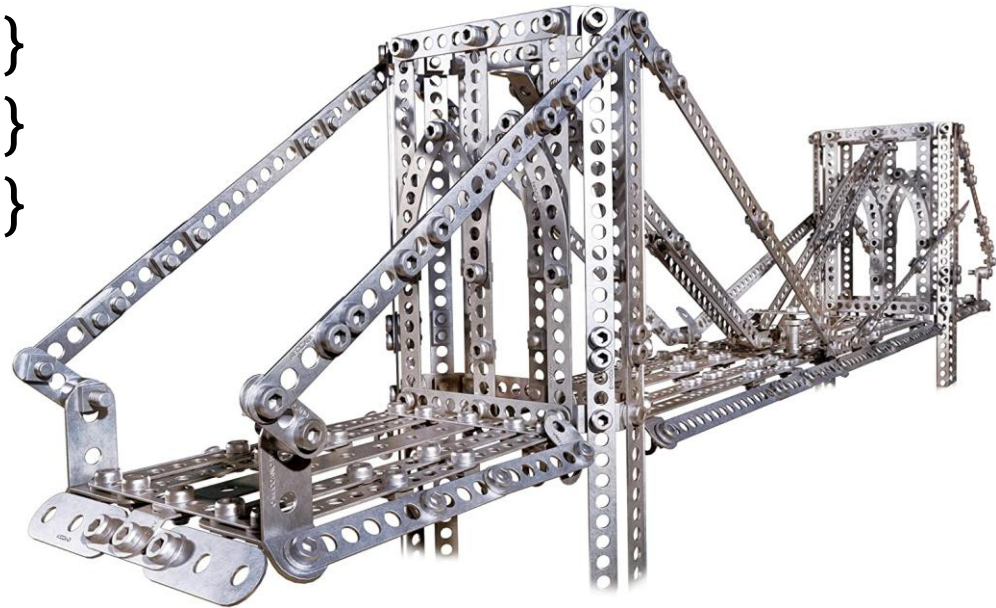
$PZ\#1 = \{x_1, y_1, \alpha_1\}$

$PZ\#2 = \{x_2, y_2, \alpha_2\}$

$PZ\#3 = \{x_3, y_3, \alpha_3\}$

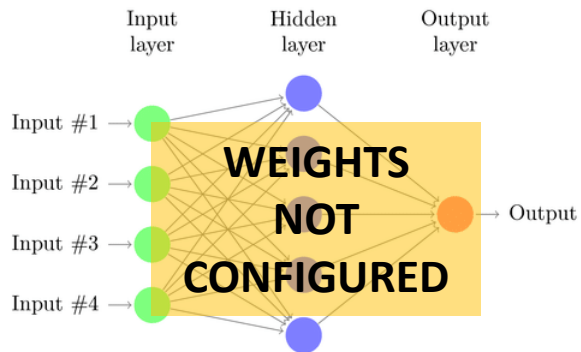
...

- The model is completed once you placed all elements
→ fixed their parameters

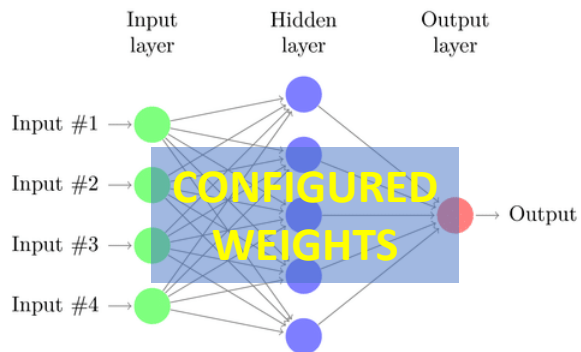


Note:
please neglect the different
types of constraints

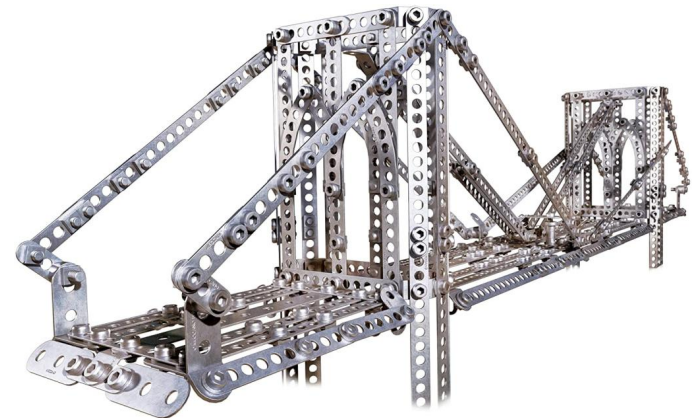
Similitude#1: weights/parameters



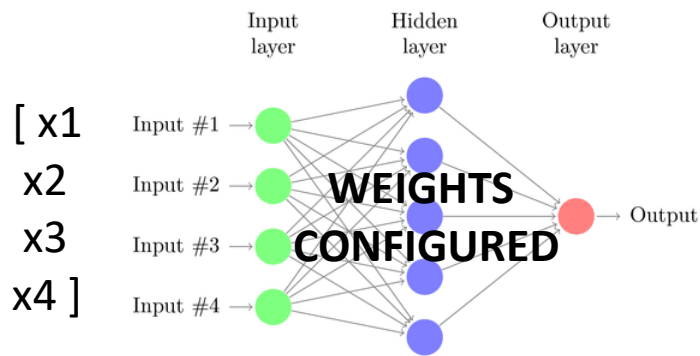
BEFORE
TRAINING



AFTER
TRAINING



Similitude#2: inputs and output, behavior

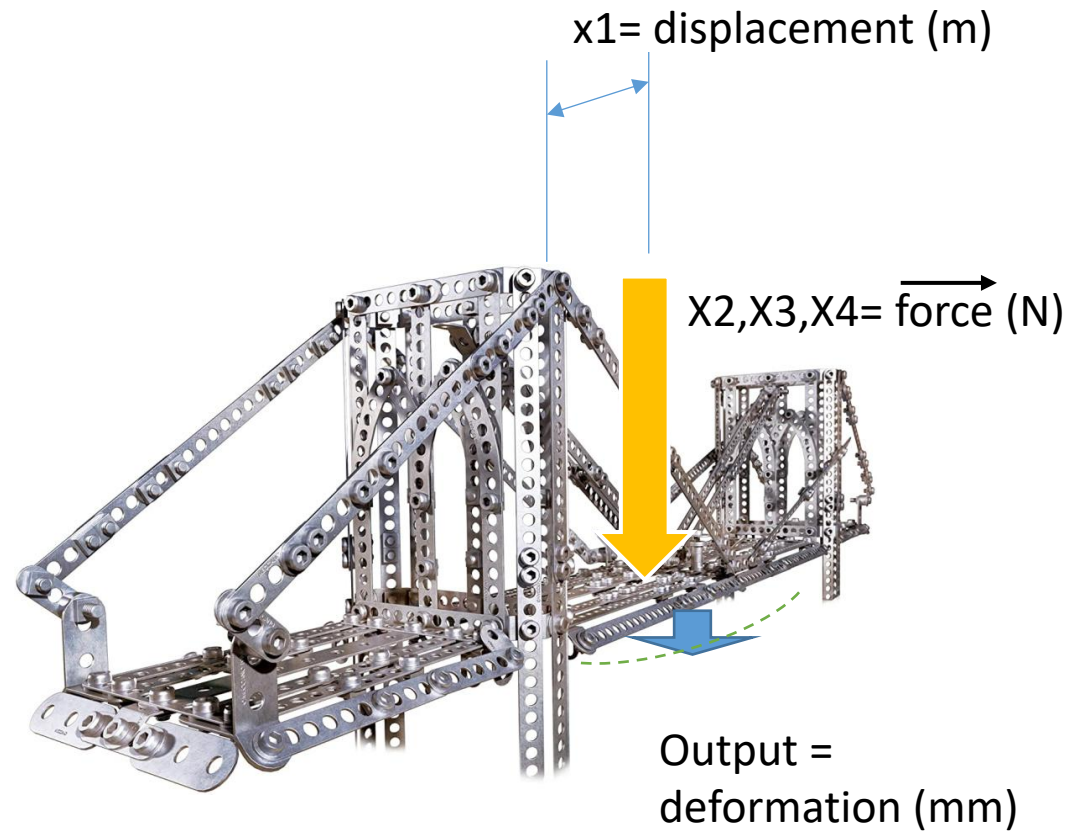


Problem1

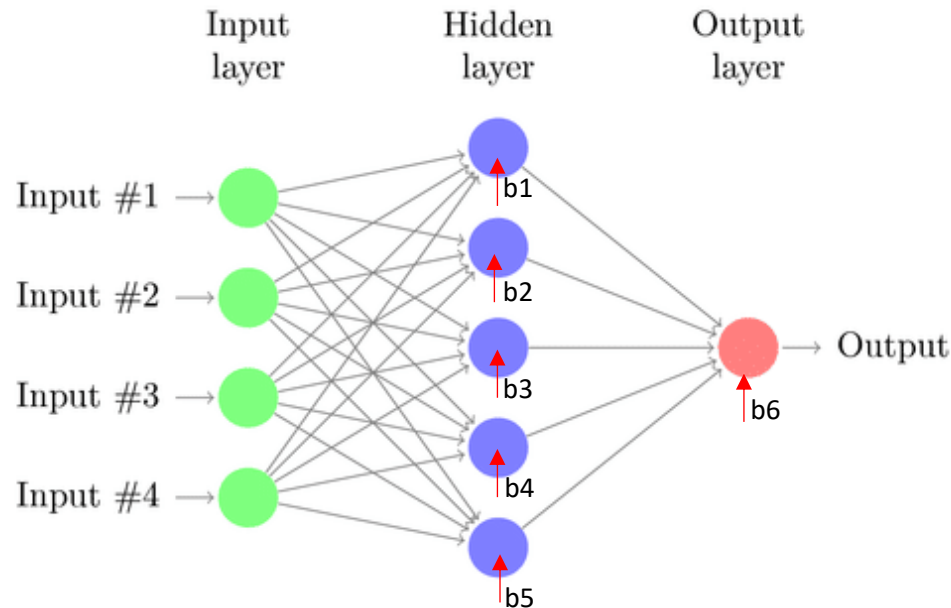
Configure weights of the neural network to correctly classify in output the inputs $[x1-x4]$ (minimum error in learning)

Problem2

Configure parameters of the elements to make a minimum deformation in output for all inputs $x1$ and $x2$ (minimum deformation in training)



Number of Parameters of NNs



↑ Bias weights

This simple neural network with 1 single hidden layer has 4×5 (hidden weights) + 5 (bias neuron) = **25** neuron weights (hidden layer), and in the final neuron we have 5 connections + 1 bias weight values
Total = $25 + 6 = 31$ parameters to be fixed

How many input data $[x_1, x_2, x_3, x_4]$ are needed in the training to fix properly the weights?

Let's see it in Colab

```
[1] import tensorflow as tf
    from tensorflow import keras
    from keras import layers

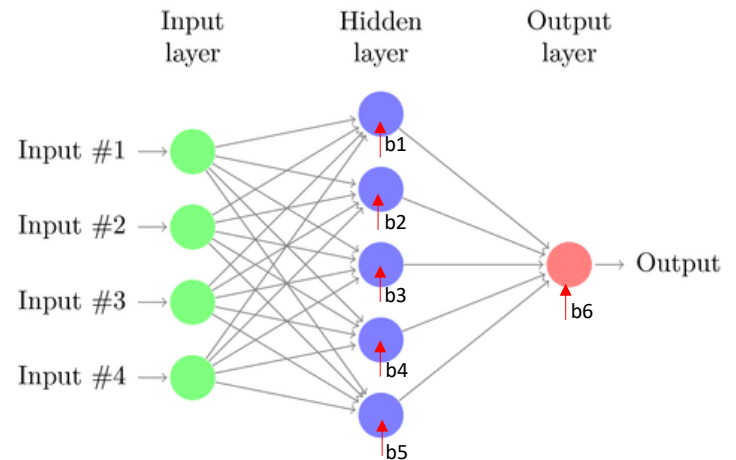
# define the 784-256-128-10 architecture using Keras
model = keras.Sequential()
model.add(layers.Dense(5, input_shape=(4,), activation="linear"))
model.add(layers.Dense(1, activation="linear"))
```

(The code will be presented
in the next lessons)

```
#print the model
model.summary()
```

Model: "sequential"

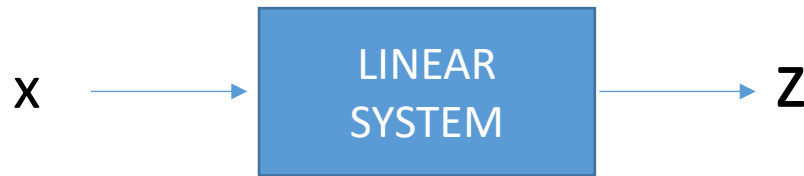
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 5)	25
dense_1 (Dense)	(None, 1)	6
Total params: 31		
Trainable params: 31		
Non-trainable params: 0		



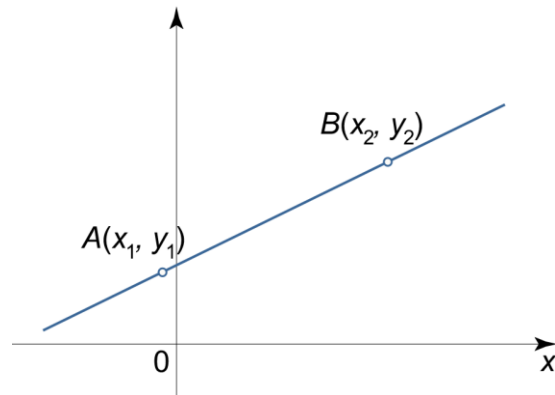
Total = 25 + 6 =
31 parameters to be fixed

How much data?

Example: a 1D linear model



$$z = \alpha x + \beta$$
$$z = w_1 x + b$$



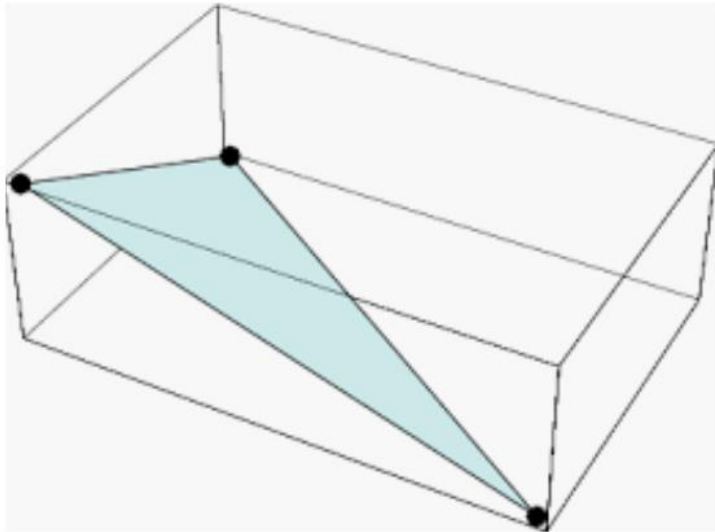
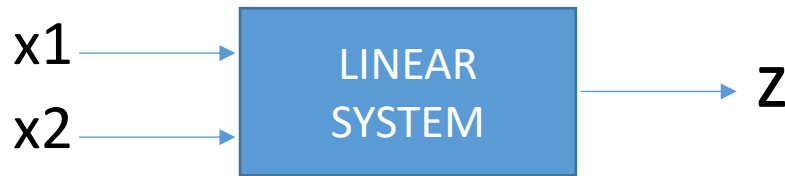
$$\text{DoF} = \#\text{Par} = 2$$

To completely describe the model
you need to fix 2 parameters:
 w_1, b .

→ You need 2 data points!

How much data?

Example: a 2D linear model



Vectorial form

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

$$z = [w_1 \ w_2] \cdot \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + b$$

DoF = #Par = 3

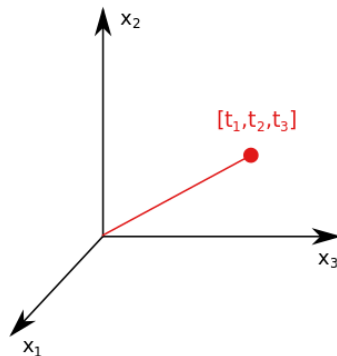
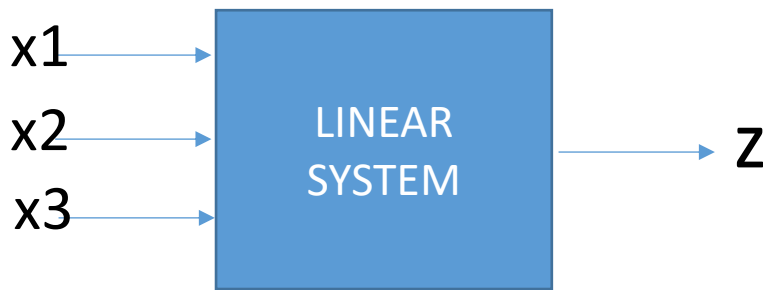
To completely describe the model
you need to fix 3 parameters:

w_1, w_2, b .

→ You need **3 data points!**

How much data?

Example: a 3D linear model



Vectorial form

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

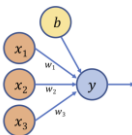
$$z = [w_1 \ w_2 \ w_3] \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + b$$

To completely describe the model
you need to fix 4 parameters:

w_1, w_2, w_3, b .

→ You need **4 data points!**

That is (almost) true also for non linear systems



By the way.... that is (also) the linear
output of a neuron

DoF in general

- The degrees of freedom for a given problem are the number of independent problem variables which must be specified to uniquely determine a solution.

- Degrees of freedom =
variables - # equations $\approx \dots$

#Par - #Data

#Inputs = 5

database = [Vectors]

#Data = 10

#Data = number of vectors in our database

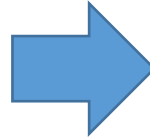
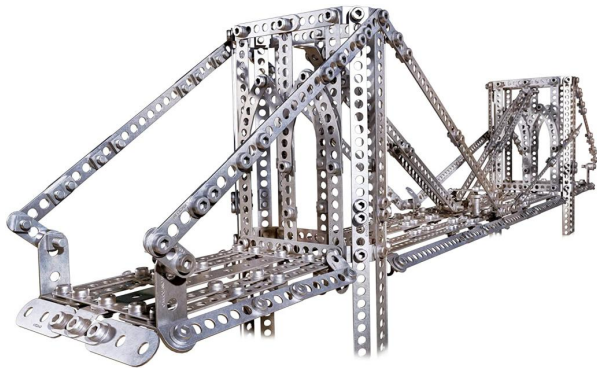
	A	B	C	D	E	F
1	x_0	x_1	x_2	x_3	x_4	y
2	0.2856096	0.333708	0.8861952	0.3166665	0.388544	0
3	0.3936127	0.9554749	0.5033009	0.48748	0.0252602	1
4	0.6678283	0.4833514	0.8402296	0.2364408	0.2593407	1
5	0.4407239	0.2374104	0.3966326	0.3496683	0.0629024	0
6	0.2622265	0.2117203	0.1967281	0.5731789	0.5462414	1
7	0.7209833	0.357229	0.5069117	0.7213588	0.7241538	1
8	0.1890057	0.1909881	0.0255648	0.7950619	0.8394897	0
9	0.7827472	0.6916138	0.956046	0.4994256	0.4466799	1
10	0.5631773	0.8218485	0.0667687	0.9893122	0.7558692	1
11	0.1593291	0.736723	0.2591398	0.4339377	0.5188055	0

Note: In complex non linear systems is not so simple, but that is a simple rule of thumb

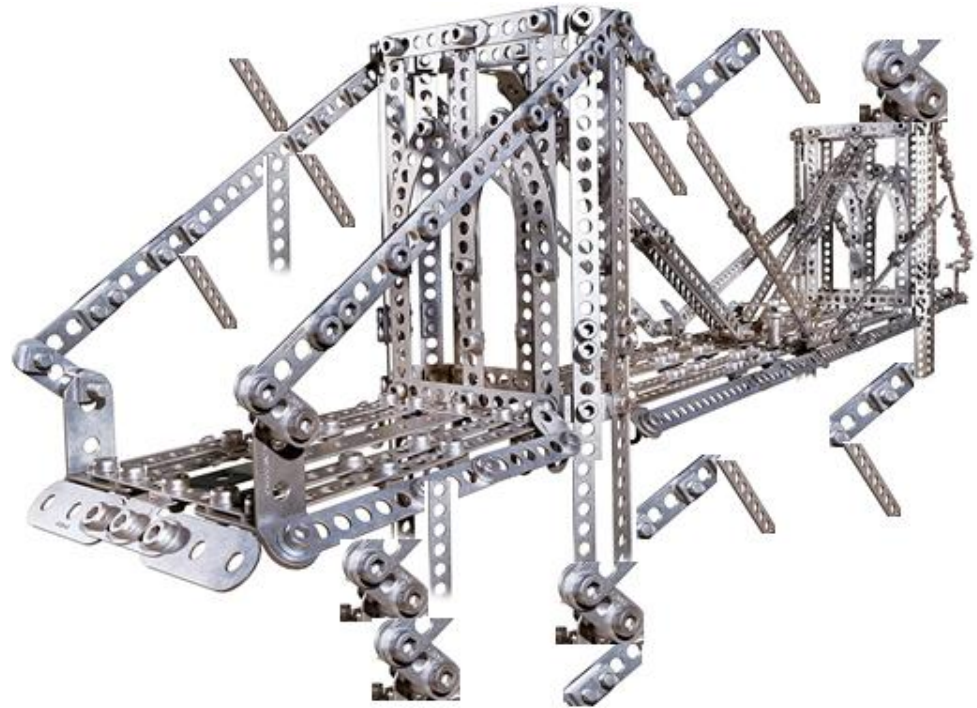
As a rough intuition, when the number of trainable parameters is too large with respect to the amount of independent and informative data, the model has excessive flexibility and may overfit.

What if... Excessive #Par (elements)

Correct #Par
(after training)



Excessive #Par
(after training)

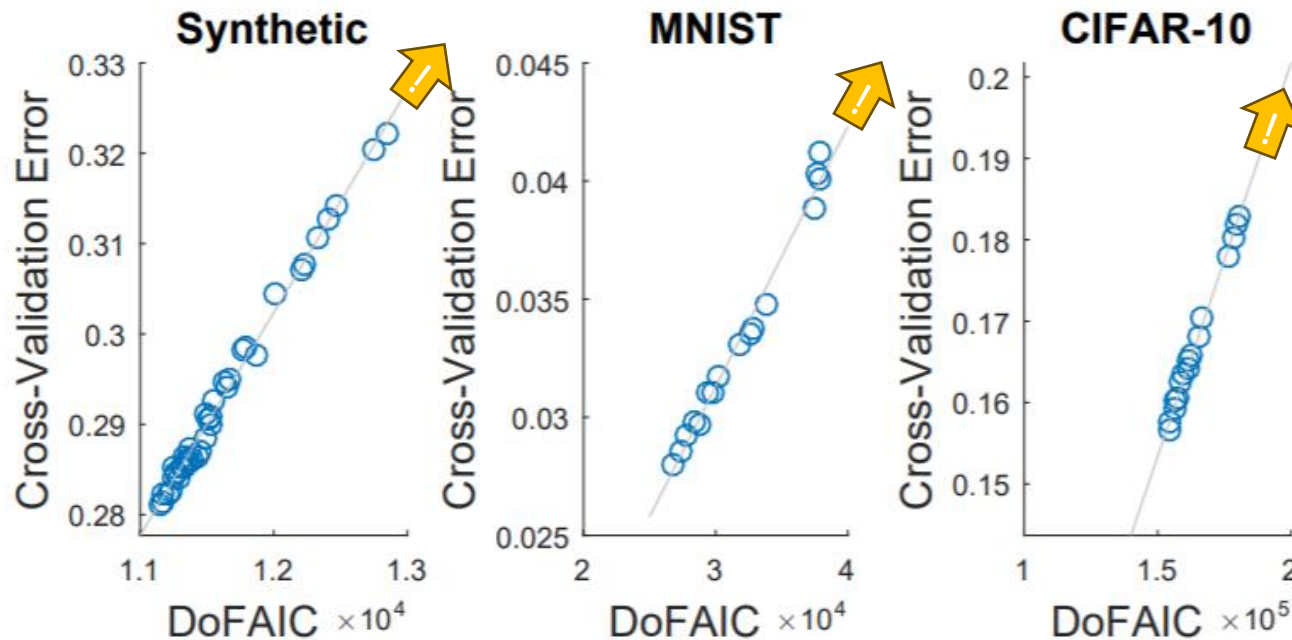


The learning method is not capable to deal with all the elements/parameters you inserted into the model with the given dataset!

The obtained model is **not optimal**, some parts or the model are **useless** or even you can get a **bad behavior**

Relationship Generalization Error VS DoF in NNs

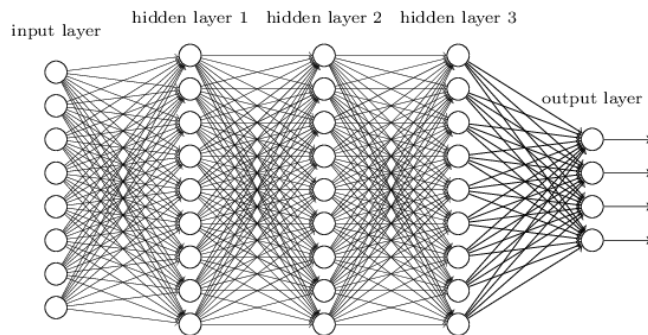
Example of generalization errors of deeplearning models in large standard classification datasets



If you leave too much «free» weights (lack of data with respect to #Par) in the network (DoF increases) the generalization capability will tend to be poor (Cross Validation Error increases)

Training large and deep learning networks

Trad. Feedforward NN



<300 parameters

■
Corr. To an image of
17x17 32bit/gray_level pixels

...Or in mm²: a normal stamp



VGGNet (2014)



138 million parameters

Corr. To an
image of
11700x11700
32bit/gray_level
pixels

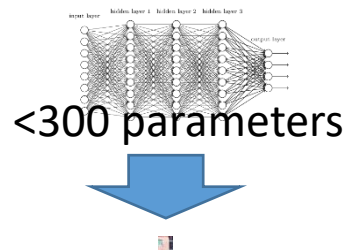


...Or in mm²: a beach volley field

#Par: an intuitive perspective

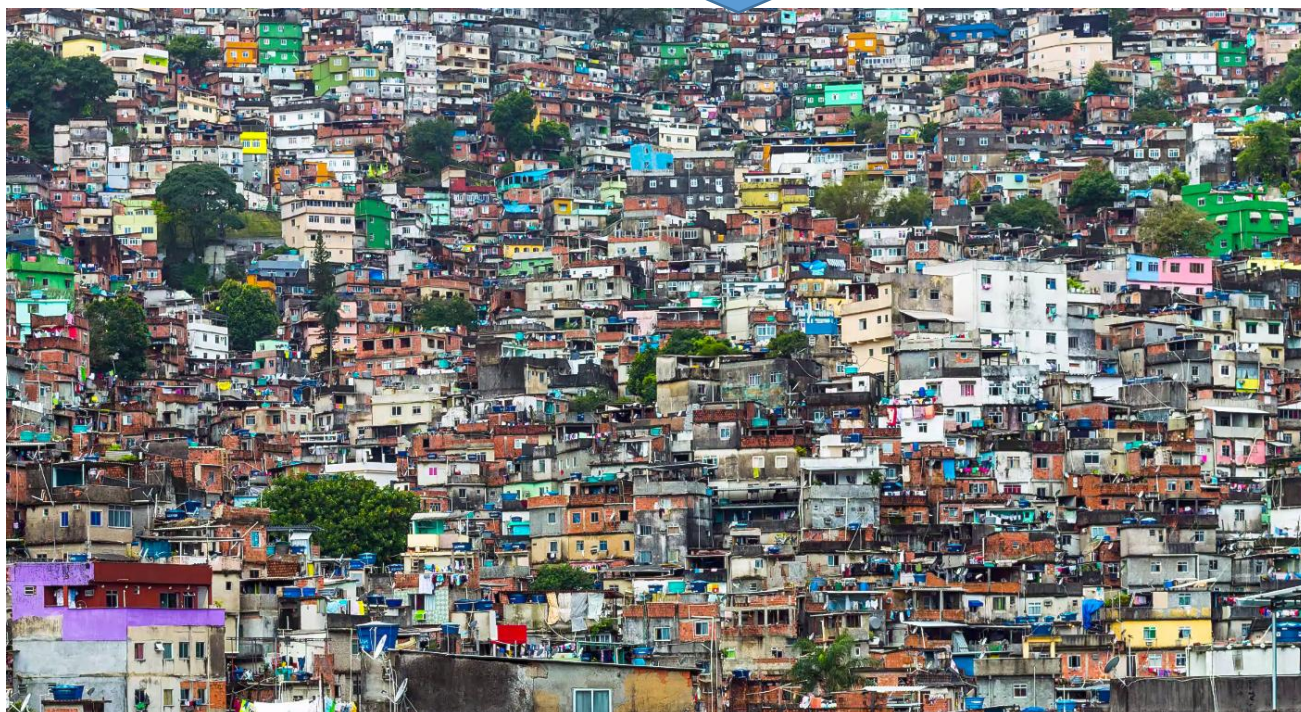
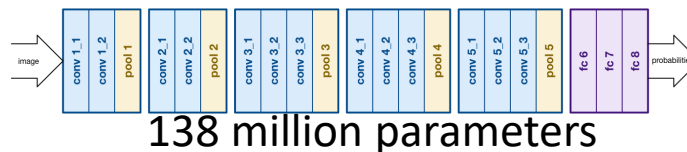
1Par=1pixel

Trad. Feedforward NN



8bit 17x17 image

VGGNet (2014)

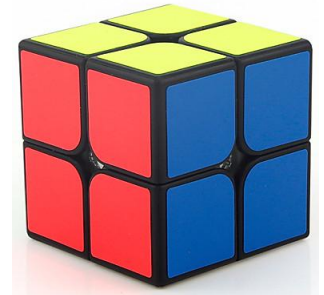


8bit 10k image < 100Mpar

In brief...

- The #Par of the model (e.g., neural network) must be carefully tuned according to
 - the size of the datasets
 - Number of vectors, Number of inputs
 - its complexity
 - Similar images? Very different examples?
- «Go deep» only if it is really necessary

Applications of the Occam's razor

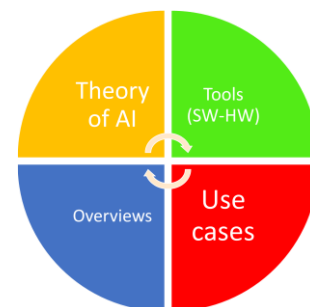




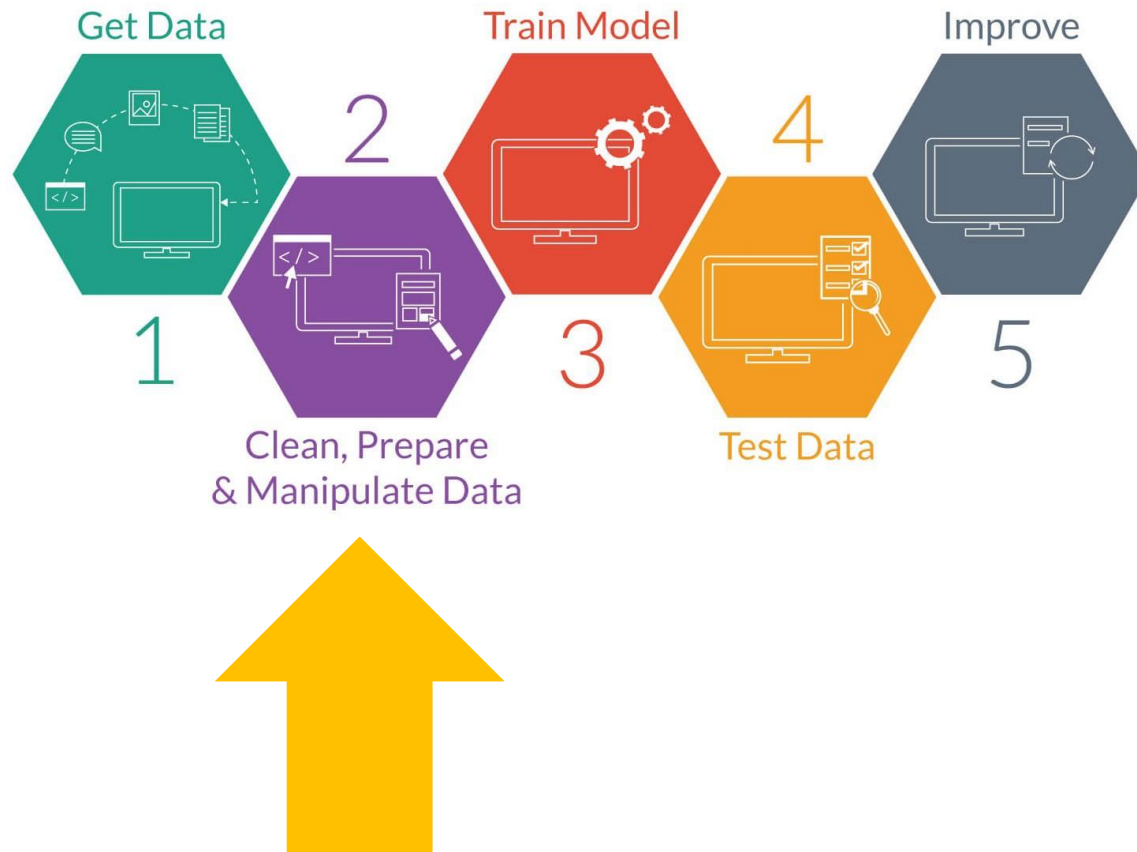
THEORY

Data leakage

One of the most relevant and limiting factor



Step 2 of the ML workflow

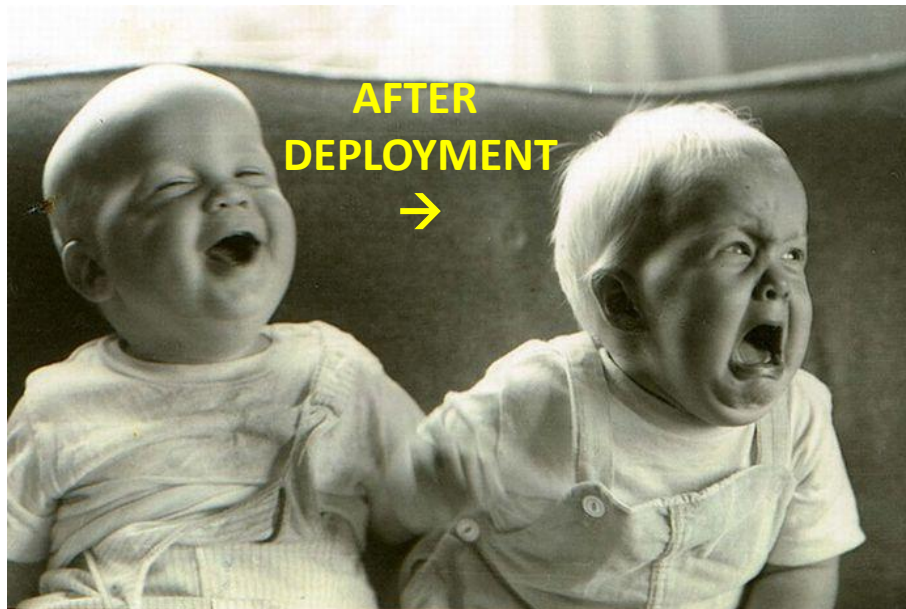


Data Leakage in Machine Learning



- Data Leakage is responsible for the **cause of invalid Machine Learning/Deep Learning model** due to the over optimization of the applied model.

**I DID A VERY
GOOD LEARNING!!
I'M GOOD AT!!**



**DAMN!!
IT OVERFITTED!!!**

Data problems in Machine Learning (2)



Two main topics

A. Data quality issues

Missing relevant features

B. Data leakage

Adding something more....



Health_0001.dat

Healthy
0.12
3,14
Bla bla bla
bla bla

Ill_0001.dat

Cancer
0.12
4,14
Bla bla bla
bla bla



```
x = dataset.drop(['iris_class'], axis=1)
```

A

Data quality issues in Machine Learning (2)



A) Missing relevant features

- For example, when we want to use a particular feature for performing **Predictive Analysis**, but that specific feature is not present at the time of training of dataset → data leakage will be introduced within the model.
- Example:
you want to add to your dataset
the concentration of OrmonX to predict CancerZ but
OrmonX is not (almost) present in the trainingHealthy datas





Data quality issues example #1

Missing something in learning data

Learning phase (measuring learning accuracy)



Dog



Dog



Dog



Cat



Cat



Cat

- What is wrong about this training/learning dataset?
- Take 1 min. to think about it...



Data quality issues example #1

Missing something in learning data

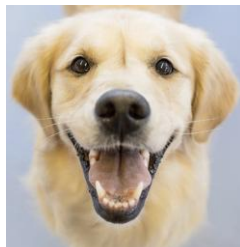
Learning phase (measuring learning accuracy)



Dog



Dog



Dog



Cat



Cat



Cat

- **Overfitting is probable**: the int. system will not learn the real differences between dogs and cats
 - Simplified rules
 - Dogs are yellow
 - Cats have ears up

Data quality issues example #1



Good generalization
test!

Missing something in learning data

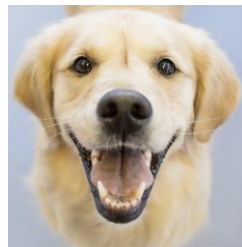
Learning phase (measuring learning accuracy)



Dog



Dog



Dog



Cat



Cat



Cat

Test phase (measuring generalization accuracy)



Dog



Dog



Dog



Cat



Cat



Cat



Data quality issues example #1

Missing something in learning data

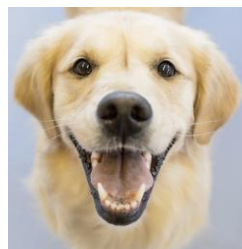
Learning phase (measuring learning accuracy)



Dog



Dog



Dog



Cat



Cat



Cat

Test phase (measuring generalization accuracy)



Dog



Dog



Dog



Cat



Cat



Cat



Data quality issues example #1

Missing something in learning data

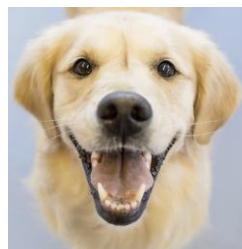
Learning phase (measuring learning accuracy)



Dog



Dog



Dog



Cat



Cat



Cat

Test phase (measuring generalization accuracy)



Dog



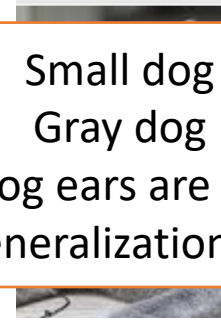
Dog



Dog



Cat



Cat



Cat

Small dog
Gray dog
Dog ears are up
→ generalization error



Data quality issues example #1

Missing something in learning data

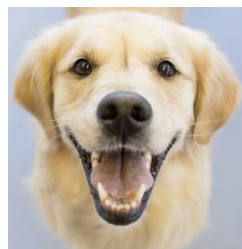
Learning phase (measuring learning accuracy)



Dog



Dog



Dog



Cat

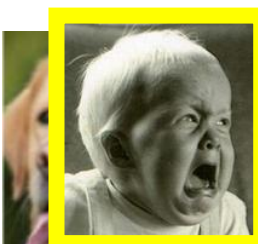


Cat



Cat

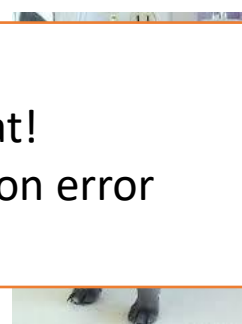
Test phase (measuring generalization accuracy)



Dog



Dog



Dog

Yellow cat!
→ generalization error



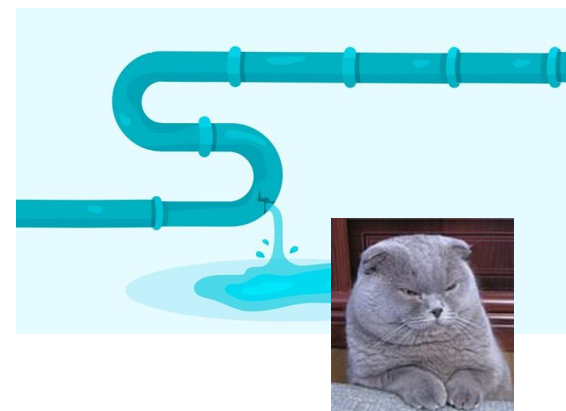
Cat



Cat



Cat



Data quality issues example #1

Missing something in learning data

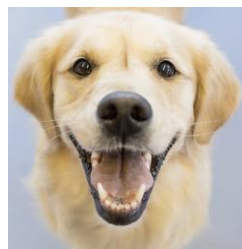
Learning phase (measuring learning accuracy)



Dog



Dog



Dog



Cat

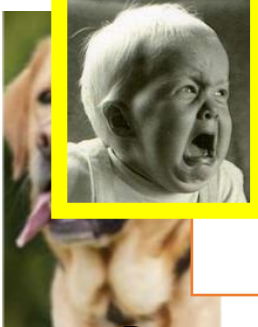


Cat



Cat

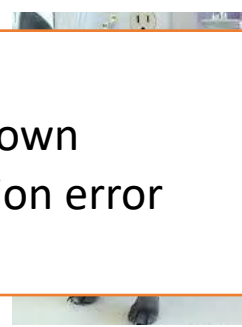
Test phase (measuring generalization accuracy)



Dog

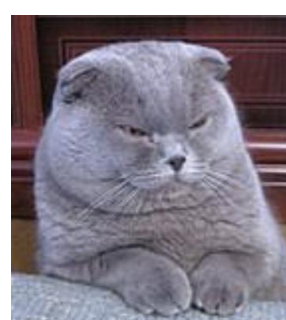


Dog



Dog

Ears are down
→ generalization error



Cat



Cat



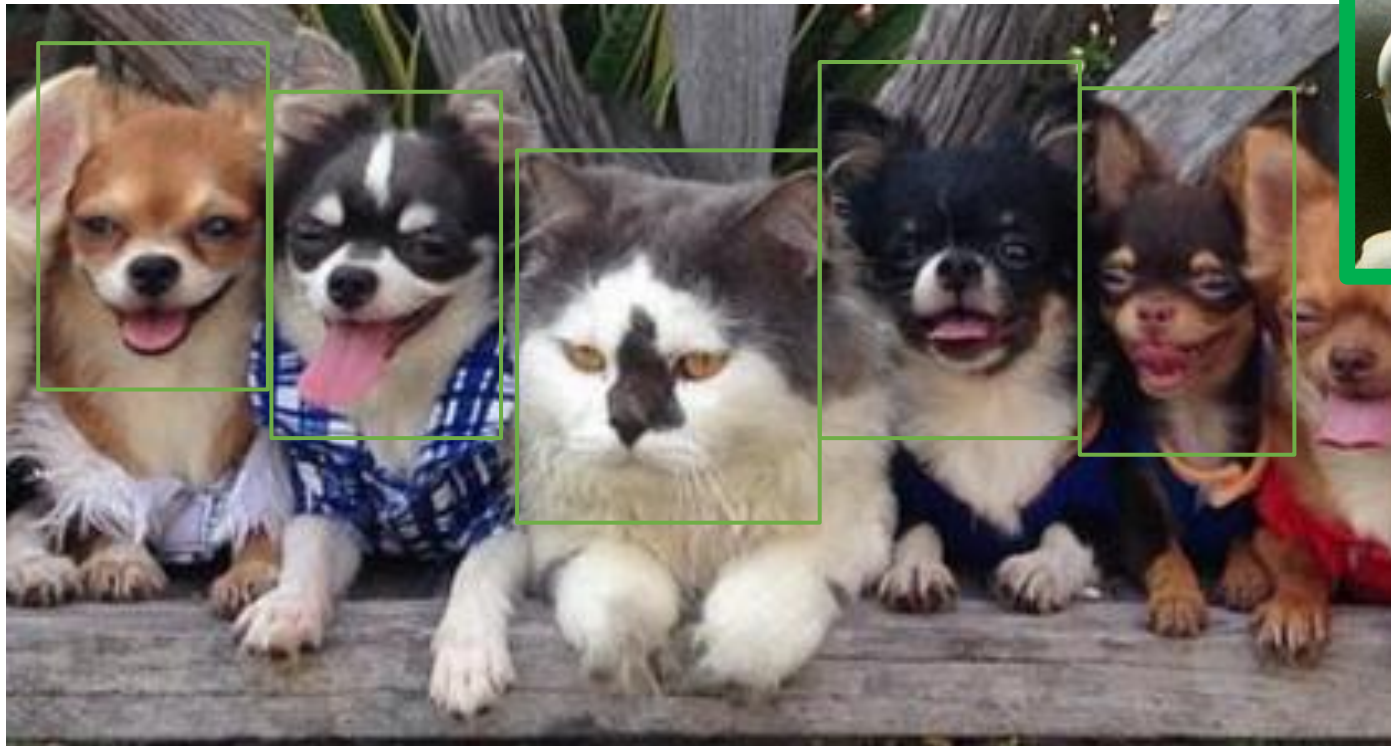
Cat

A

Data quality issues example #1



You need **more good “salient” images** to let the intelligent systems understand the what are the important features to distinguish a cat from a dog



Data Leakage in Machine Learning



This is one of the most sneaky... probably the first case of critical data leakage

B) Adding something more ...

- When information from outside the “expected” training information in the dataset is used to create the model.
 - This additional information can allow the model to learn or know something that it otherwise would not know and in turn invalidate the estimated performance of the model being constructed.
- This additional learning of information by the applied model will disapprove the computed estimated generalization performance of the model
 - your performance estimation of the model once it will be deployed tends to be too much optimistic!

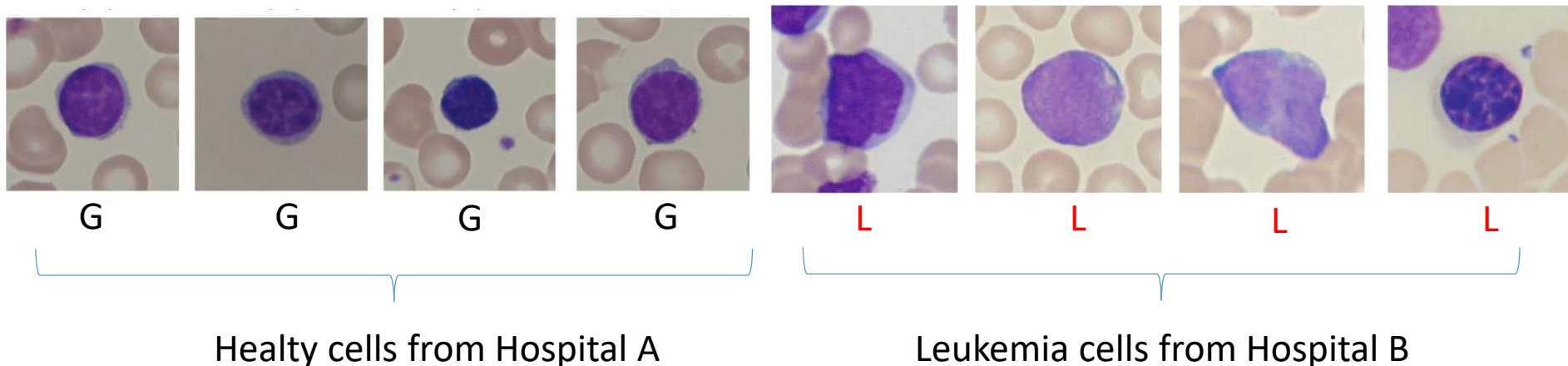
Data Leakage example #2



- Something you shouldn't know

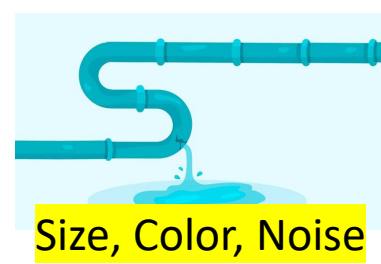
Example: **Good/Leukemia** white cell images

Learning phase (measuring learning accuracy)



What is wrong with the dataset?

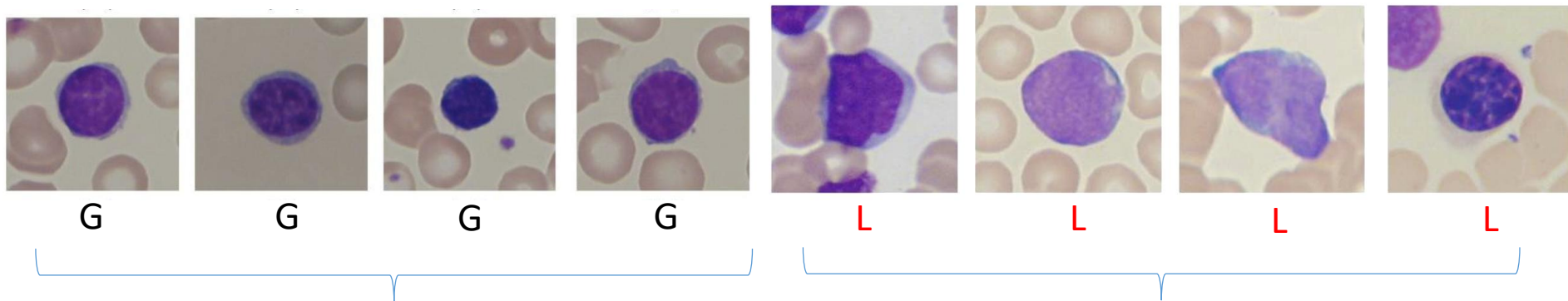
Data Leakage example #2



- Something you shouldn't know

Example: good/leukemia white cell images

Learning phase (measuring learning accuracy)



Healthy cells from Hospital A

Leukemia cells from Hospital B

- Images are slightly larger!
- Images are lighter in color
- Noise level is different

The AI model can learn to check only this features of the images (→ Hospital B = cancer) rather than the shapes...

Data Leakage can happen!



B1) The Leakage of data
from test dataset → to training data set

- Data specifically intended to check the generality of the model is spilled in the train dataset....
- Vice versa is slightly better, but still not ideal

Check this portions/chunks of data in Train (T) and Validation-Test (V)

- SLIGHTLY BAD leakage:

$[T1, T2, T3] + [V1, V2, V3] \rightarrow \text{leakage} \rightarrow [T1, T2, T3] + [V1, V2, V3, T3]$

Some information you designed to be used only in validation, is now in training...

- **VERY BAD leakage:**

$[T1, T2, T3] + [V1, V2, V3] \rightarrow \text{leakage} \rightarrow [T1, T2, T3, V1] + [V1, V2, V3]$

The V1 section of the test is not anymore unseen

Data Leakage can happen!



B2) Leakage of future data into the past data

B3) Usage of data outside the scope of the applied algorithm

Example:

Dataset of adults → Deployment of a classifier for diagnosis used also for kids

Panel 2: Typical adult reference ranges for thyroid function tests

TSH	0.5–5.5mIU/L
T4	60–135nmol/L
Free T4	9.4–25pmol/L
T3	1.1–2.8nmol/L
Free T3	3.0–8.6pmol/L

Data Leakage example #3

Can you see where is the problem?

Learning phase (measuring learning accuracy)



Dog



Dog



Dog



Cat



Cat



Cat

Test phase (measuring generalization accuracy)



Dog



Dog



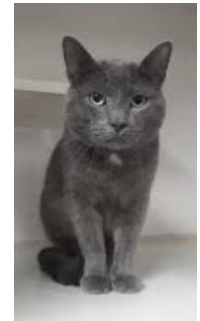
Dog



Cat



Cat



Cat

C Data Leakage example #3

Statistical independence is relevant!

The reuse of samples
(or cropped part of them),
or the reuse of the same users,
produces poor generalization

- The “Déjà vu” (warning, it is just a mnemonic reference...)

Learning phase (measuring learning accuracy)



Dog



Dog



Dog



Cat



Cat



Cat

Test phase (measuring generalization accuracy)



Dog



Dog



Dog



Cat



Cat



Cat

Data Leakage can happen!



In brief, we have two primary sources of data leakage in Machine Learning algorithms:

- A. Feature attributes**
(variables are saying too much...)
- B. Training data set**
(chunk of data used in the wrong phase)

Time series: a special case for Regression and Classification

Time Series Prediction

Definition: Estimating **future** values based on **past data**.



Time Series Classification

Definition: Assigning **labels** based on **past data**.



Here it is the problem:

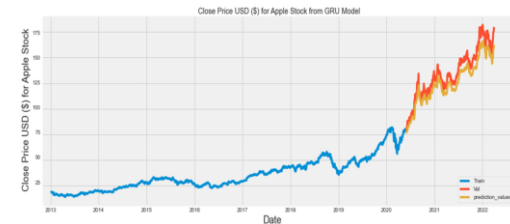
in both cases, the model can be effective only if the past data is consistent (not mixed/scrambled) in

- training (input to the learning method)
- inference (input to the model)

Time series: a special case for Regression

Examples:

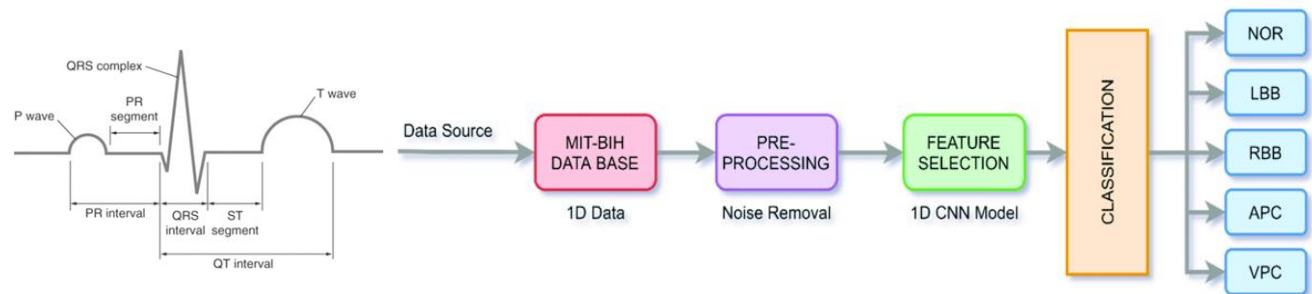
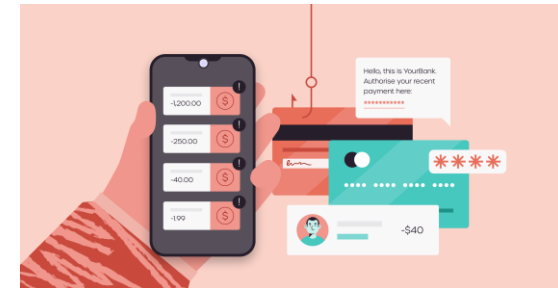
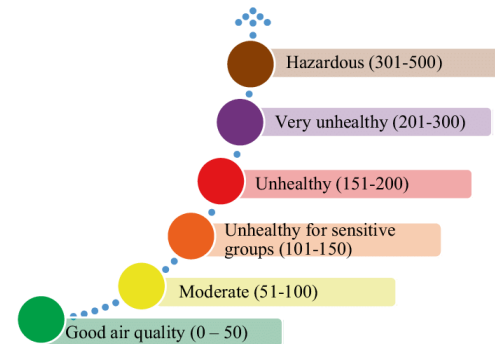
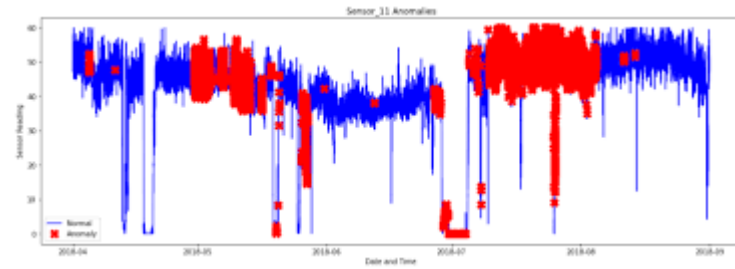
- **Industry:** Predicting machine failures in predictive maintenance.
- **Supply Chain:** Forecasting product demand.
- **Environment:** Weather forecasting, temperature prediction.
- **Finance:** Stock price forecasting, economic trends.
- **Healthcare:** Patient vital sign prediction.

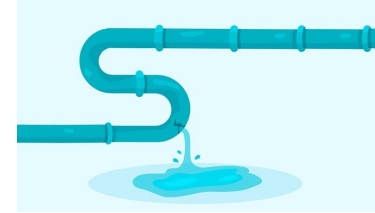


Time series: a special case for Classification

Examples:

- **Industry:** Detecting anomalies in sensor data.
- **Supply Chain:** Identifying demand patterns for product categories.
- **Environment:** Classifying air quality levels based on sensor data.
- **Finance:** Fraud detection in transaction sequences.
- **Healthcare:** ECG signal classification for heart disease detection





Checking the presence of Data Leakage

Data Leakage is observed in **time-related complex** datasets such as:

- Dividing time series dataset into training and test, the dataset can be an error-prone problem
- **Be very careful!**

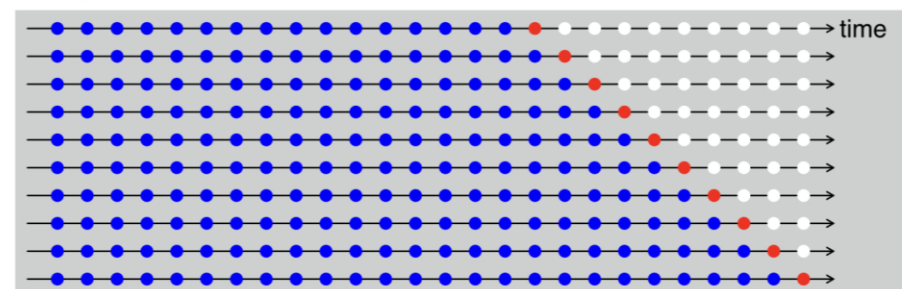
Split 1:	Test set	Training set		
Split 2:	Training set	Test set	Training set	
Split 3:	Training set		Test set	Training set
Split 4:	Training set			Test set
Split 5:	Training set			
	Time 1	Time 2	Time 3	Time 4

Here the time continuity is not always guaranteed (only in the “short term”). Example, in the first split the validation data is in the past w.r.t. the training, in the second split the training data has a temporal discontinuity, etc..

Traditional evaluation



Time series cross-validation

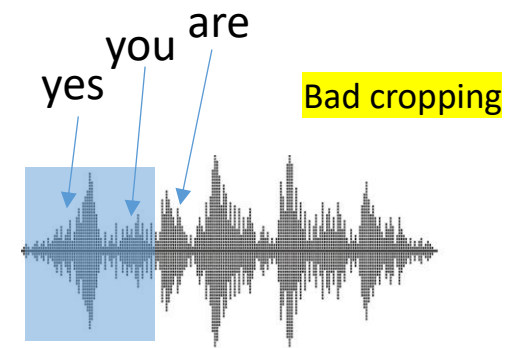


Time continuity is fully guaranteed

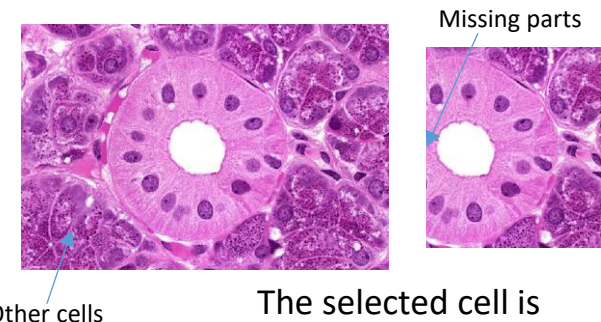
Checking the presence of Data Leakage

Data Leakage is observed in time-related complex datasets such as:

- Storage of analog observations in the form of audios and images in separate files having a **defined size and timestamp**
- Implementation of sampling in a **graphical problem** is a complex task (e.g., slicing an image)



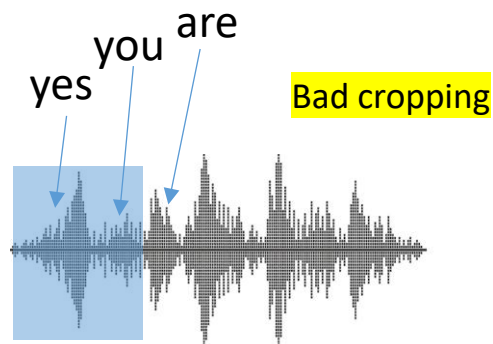
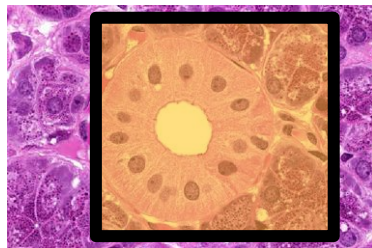
The selected frame for the training of the word «YES» is too long and contains also «YOU»



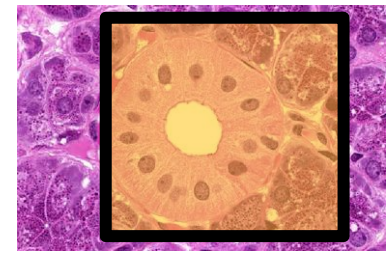
Data Leakage: the “cropping” problem

A Widespread Issue

- The **cropping problem** is a form of **data leakage** that affects multiple types of datasets and applications. It occurs when a model learns patterns from **cropped** or **contextually biased** data, leading to misleadingly high performance but poor generalization.

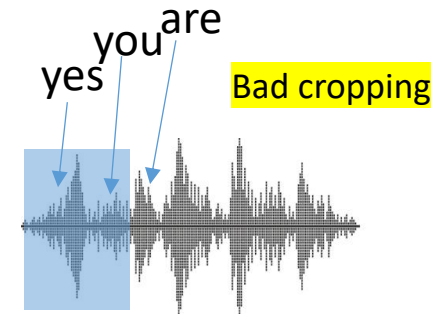


Data Leakage: the “cropping” problem



Cross-Domain Examples:

- **Images:** A model trained to classify objects may unintentionally rely on **cropped edges** or **artifacts** from image preprocessing rather than actual object features.
- **Audio:** A speech recognition model may learn background noise patterns from cropped samples instead of focusing on spoken words.
- **Structured Data:** In medical diagnosis, if training data is cropped to contain only extreme cases, the model might fail on intermediate cases.
- **Unstructured Data (Text):** A sentiment analysis model trained on social media posts might learn to rely on specific truncated phrases instead of full context.



GNU Solidario Hospital
Autvía del Norte 12485
Las Palmas de Gran Canaria
Spain

LABORATORY REPORT			
Name	Ana Betz	Patient ID	PAC001
Date	2011-08-25 08:32	Age	25y 10m 26d
Doctor	Carmen Cortada	Sex	Female
		Test id	B165A4F4
COMPLETE BLOOD COUNT			
Test Name	Result	Normal Range	Units
Hemoglobin	12	11.0 - 16.0	g/dL
HCT	3.3	3.5-5.50	10 ¹² /uL
HCT	36	37.0-50.0	%
MCH	83	83-95	f
MCH	28	27-31	pg
MCHC	33	32.0-36.0	g/dL
RDW-CV	12	11.5-14.5	%
RDW-SD	44	35-56	f
WBC	6.7	4.5-11	10 ⁹ /uL
NEU%	60	40-70	%
LYM%	30	20-45	%
MON%	8	2-10	%
EOS%	2	1-6	%
LYM#	2	1.5-4.0	10 ⁹ /uL
GRAN#	4.7	2.0-7.5	10 ⁹ /uL
PLT	256	150-450	10 ⁹ /uL
ESR	2	Up to 15	mm/hr

"Nice store. Seems **clean** and **well organized**. It is a bit on the **smaller** size as far as other stores go, so prepare for it to perhaps feel a little **crowded**, even if there aren't that many people in the store. This also means that their stock of certain items may be a bit **smaller** than the **larger** locations, but that's to be expected."

Nice
Clean
Well
Organized
Smaller
Crowded
Larger



Minimizing Data Leakage

Cropping

Recalculation of required data using cross-validation

Carefully

Dividing the Dataset



Feature Selection



Outlier detection & removal



Projection Method



Scaling of selected features



Training Dataset



Test Dataset

Avoid
Unwanted
features

Cm, mm

Main points

- Using datasets without previous activities is useless dangerous and it wastes your time
- Number of degrees of freedom /parameters
- **Data Leakage!**

